

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY  
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



DEEP LEARNING AND ITS APPLICATIONS  
SEMESTER 261

## Assignment 2 - Proposal

**Faculty:** Computer Science & Engineering

**Instructor:** Le Thanh Sach

**Class Number:** CC01

**Group:** G-M10

**Date of Submission:** 7/10/2026

| Group Member   | ID Student | Contribution |
|----------------|------------|--------------|
| Tran Gia Lam   | 2352670    | 100 %        |
| Nguyen Huu Cau | 2352129    | 100 %        |



## Contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Problem Definition and Objectives</b>                    | <b>3</b>  |
| <b>2</b> | <b>Proposed Methods and Evaluation</b>                      | <b>6</b>  |
| 2.1      | Lightweight Baseline                                        | 6         |
| 2.2      | Pretrained Model                                            | 6         |
| 2.3      | Training Protocol                                           | 6         |
| 2.4      | Controlled Experiment: Frozen Backbone vs. Full Fine-Tuning | 6         |
| 2.5      | Evaluation and Error Analysis                               | 7         |
| <b>3</b> | <b>Compute Plan and Expected Challenges</b>                 | <b>8</b>  |
| 3.1      | Compute Estimate                                            | 8         |
| 3.2      | Expected Challenges                                         | 8         |
| <b>4</b> | <b>Team Responsibilities and Milestones</b>                 | <b>9</b>  |
| 4.1      | Current Progress and Planned Work                           | 9         |
| 4.2      | Milestone Plan                                              | 9         |
| <b>5</b> | <b>AI Usage Disclosure and References</b>                   | <b>10</b> |
| 5.1      | Ai Usage Disclosure                                         | 10        |
| 5.2      | References                                                  | 10        |



## Workload

| Member         | Workload                                                                                                                                                                       | Contribution |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| Tran Gia Lam   | Write Sections 1, 5–8; check consistency with Cau's EDA; integrate the report and references; update GitHub Pages; manage submission.                                          | 100 %        |
| Nguyen Huu Cau | Write Sections 2–4; finalize duplicate grouping and split files; synchronize notes, CSV/JSON review records, figures, and checksums; provide the executed notebook and output. | 100 %        |

## 1 Problem Definition and Objectives

Object detection identifies objects and locates them in an image. This project will use PASCAL VOC 2007 and its 20 object classes [1]. The input is an RGB image. The output is a set of detections, each containing a class label, a confidence score, and a bounding box in image coordinates. The box is written as  $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ . An image may contain several objects from different classes.

The main goal is to build a reproducible training and evaluation pipeline. We will compare a lightweight detector with a pretrained Faster R-CNN model, then study the effect of freezing its backbone. The comparison will cover detection quality, training cost, and inference time.

The preliminary EDA shows uneven class counts and a wide range of object sizes. These findings motivate two questions: how well does each model detect less frequent classes, and what kinds of objects are commonly missed? Class-level AP and examples of detection errors will help answer them. No model scores have been measured for this proposal.

Main experiments will begin after instructor approval and after the reviewed split is finalized. All runs will use the same saved train and validation image IDs.



ha



ha

## 2 Proposed Methods and Evaluation

### 2.1 Lightweight Baseline

The baseline will be SSDLite320 with a MobileNetV3-Large backbone, using the Torchvision implementation. It takes images at  $320 \times 320$  pixels [2]. We plan to initialize the backbone with ImageNet weights and train new detection heads on VOC2007. The output will cover the 20 VOC classes and one background class. This baseline gives us a starting point for measuring the accuracy and cost of a compact detector.

### 2.2 Pretrained Model

The main model will be Faster R-CNN with a ResNet-50 backbone and a Feature Pyramid Network (FPN), initialized with Torchvision's COCO\_V1 detection weights [3]. Its final box predictor will be replaced for the 20 VOC classes and background, following Torchvision's fine-tuning approach [4]. The planned resize uses a shorter image side of 640 pixels and a maximum longer side of 1000 pixels while keeping the aspect ratio.

This comparison covers complete model configurations, including their initialization and input scale. We will record these differences when discussing accuracy and cost. The controlled experiment below will use the same Faster R-CNN model in both conditions.

### 2.3 Training Protocol

The initial plan is 20 epochs per run, using SGD with momentum 0.9 and weight decay  $5 \times 10^{-4}$ . The starting learning rates are 0.01 for SSDLite and 0.005 for Faster R-CNN, reduced by a factor of ten after epochs 12 and 16. These are proposed settings. A short pilot will check memory use and stable loss values before the main runs; any changes will be recorded.

Training augmentation will use random horizontal flipping with probability 0.5, with the boxes updated accordingly. Validation images will use deterministic preprocessing. Each model's internal transform will handle resizing and normalization. Valid non-difficult boxes will be used as training targets. Validation annotations will retain the difficult flags so the evaluator can apply the VOC ignore rules.

We will use seed 42 for the initial study and save the configuration, package versions, split IDs, loss logs, and predictions. Validation mAP will be computed after each epoch. The checkpoint with the highest validation mAP will be selected; a tie will keep the earlier checkpoint.

### 2.4 Controlled Experiment: Frozen Backbone vs. Full Fine-Tuning

The hypothesis is that updating the ResNet-50 feature extractor may improve detection on VOC2007, with additional training cost. Two Faster R-CNN runs will test this:

- **Frozen backbone:** keep the ResNet-50 feature extractor fixed; train the FPN, region proposal network, and prediction heads.

- **Full fine-tuning:** update all trainable model parameters, including the ResNet-50 feature extractor. The model's normalization configuration will stay the same.

Both runs will start from the same pretrained weights and the same initial VOC predictor. Split IDs, seed, resizing, augmentation, effective batch size, optimizer, learning-rate schedule, epoch limit, and evaluation settings will be fixed. The pilot settings will be finalized before either run begins.

We will report the change in validation mAP, AP for each class, training time, and peak GPU memory. A higher mAP will indicate an improvement in this experiment. Its size and compute cost will be discussed together. The initial comparison uses one seed, so it will not establish how consistent the difference is across repeated runs.

## 2.5 Evaluation and Error Analysis

The primary metric will be VOC2007 11-point interpolated mAP at an IoU threshold of 0.5. AP will also be reported for each of the 20 classes [5]. The evaluator will follow the official VOC matching and difficult-object rules [1]. Its IoU calculation and AP values will be checked against the devkit using small test cases before use.

Inference measurements will use batch size one and the same sample of validation images on the same GPU. We will warm up each model, synchronize CUDA around timing, and report mean latency with the resize and postprocessing included. Disk loading will be measured separately. Score filtering, non-maximum suppression settings, and detection limits will be recorded and held fixed within the freezing study.

The error analysis will inspect missed objects, wrong class labels, inaccurate boxes, and duplicate detections. Each example will show both predictions and ground-truth boxes. We will relate these errors to object size, class frequency, and crowded scenes. The official test set is reserved for final evaluation after settings are selected on validation data; overlap checks will be completed before it is used.

### 3 Compute Plan and Expected Challenges

#### 3.1 Compute Estimate

The target environment is Google Colab with one NVIDIA A100 GPU. We will check the assigned GPU before training and record its model, available GPU memory, CUDA, PyTorch, and Torchvision versions at the start of each run. Mixed precision will be enabled if the pilot confirms that training is stable. Checkpoints will include the model, optimizer, scheduler, and scaler states so training can resume across sessions. Colab hardware availability and session limits can vary [6]. Any change of GPU will be recorded with the run configuration.

The initial workload is three runs of 20 epochs. For planning, approximately 4000 training images give the following estimates. Exact counts will come from the finalized split.

| Run                            | Epochs | Batch size | Training steps  |
|--------------------------------|--------|------------|-----------------|
| SSDLite baseline               | 20     | 8          | $\approx 10000$ |
| Faster R-CNN, frozen backbone  | 20     | 2          | $\approx 40000$ |
| Faster R-CNN, full fine-tuning | 20     | 2          | $\approx 40000$ |

The proposed allowance is 24 A100 GPU-hours for the main runs and 6 additional GPU-hours for checks, evaluation, and interrupted runs. This 30-hour allowance is a planning budget. Training speed and memory use have not been measured on A100 for these configurations. The pilot will check whether the allowance is enough and provide a revised time estimate before the main runs.

After approval, a pilot of 200 training batches and one full validation pass per configuration will measure runtime and peak GPU memory on A100. The training batches will be averaged after warm-up. The per-run estimate, in seconds, is

$$T_{\text{run}} \approx E \left( \left\lceil \frac{N_{\text{train}}}{B} \right\rceil t_{\text{step}} + t_{\text{val}} \right),$$

where  $E$  is the epoch count,  $B$  is batch size,  $t_{\text{step}}$  is the measured average training-step time, and  $t_{\text{val}}$  is the validation time per epoch. Data preparation and checkpoint saving will be added separately. If memory requires a smaller micro-batch, gradient accumulation will keep the Faster R-CNN effective batch size unchanged in both controlled runs.

#### 3.2 Expected Challenges

Small boxes may lose useful detail after resizing, and less frequent classes provide fewer training examples. Crowded images can also produce missed or duplicate detections. We will examine these cases in the class-level results and error examples before deciding whether extra augmentation or sampling changes are useful. GPU memory and session interruptions are practical limits; the pilot and resumable checkpoints will guide the final run schedule.

## 4 Team Responsibilities and Milestones

### 4.1 Current Progress and Planned Work

The preliminary EDA handoff contains an executed notebook, tables, figures, split IDs, and written observations. The duplicate review identified one near-duplicate pair across train and validation. The split and exported review records still need to be finalized. Instructor approval is not recorded in the supplied materials, and no A2 detector has been trained in the reviewed run.

For M1, Tran Gia Lam will write Sections 1, 5, 6, 7, and 8, integrate the proposal, update the assignment page and repository, and manage submission. Nguyen Huu Cau will write Sections 2–4 and finalize the EDA, duplicate review, split records, and exported evidence. M1 responsibilities are listed in the Workload table. The table below shows the planned work after approval.

| Member                 | After approval                                                                                                                      |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Tran Gia Lam<br>Leader | Implement Faster R-CNN and the frozen/full fine-tuning study;<br>coordinate the shared evaluator, A100 profiling, and final report. |
| Nguyen Huu Cau         | Implement and train the SSDLite baseline; review its predictions;<br>contribute to class-level results and error analysis.          |

Both members will review the split, metric definitions, and reported results. Completed contributions will be recorded as the work is finished.

### 4.2 Milestone Plan

The schedule follows the A2 milestones in the course handbook [7]. All milestone deadlines are at 23:59 Vietnam time (GMT+7), unless the LMS states otherwise.

| Milestone      | Due date    | Planned deliverables                                                                                                                  |
|----------------|-------------|---------------------------------------------------------------------------------------------------------------------------------------|
| M1 Proposal    | 7 Oct 2026  | Dataset and task proposal, preliminary EDA, split and leakage plan, metrics, model plans, and compute estimate.                       |
| After approval | Before M2   | Finalize the split; verify the data loader and evaluator; run pilots; train the baseline and start Faster R-CNN fine-tuning.          |
| M2 Draft       | 28 Oct 2026 | Baseline, pretrained model in progress, EDA, preliminary metrics, and updated run evidence.                                           |
| M3 Final       | 11 Nov 2026 | Completed comparison and controlled study, error and compute analysis, final report, slides, video, assignment page, and checkpoints. |

Each run will use a separate output folder. The notebook, configuration, split IDs, checkpoints, logs, and predictions will be linked from the assignment page so the report can be checked against the actual outputs.

## 5 AI Usage Disclosure and References

### 5.1 Ai Usage Disclosure

The team used AI to develop ideas, review source code and improve the report. AI did not replace running the experiments or generate the reported experimental results. Team members check the results, interpretations and final content before submission.

### 5.2 References

- [1] PASCAL VOC, *The PASCAL Visual Object Classes Challenge 2007 Development Kit*. Task definitions, annotations, and evaluation rules. Available at: <https://www.robots.ox.ac.uk/~vgg/projects/pascal/VOC/voc2007/htmldoc/index.html>.
- [2] Torchvision, *SSDLite320-MobileNetV3-Large Model Documentation*. Available at: [https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.detection.ssd\\_lite320\\_mobilenet\\_v3\\_large.html](https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.detection.ssd_lite320_mobilenet_v3_large.html).
- [3] Torchvision, *Faster R-CNN ResNet-50-FPN Model Documentation*. Available at: [https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.detection.fasterrcnn\\_resnet50\\_fpn.html](https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html).
- [4] PyTorch, *TorchVision Object Detection Finetuning Tutorial*. Available at: [https://docs.pytorch.org/tutorials/intermediate/torchvision\\_tutorial.html](https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html).
- [5] M. Everingham et al., “The PASCAL Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, pp. 303–338, 2010. Available at: <https://www.robots.ox.ac.uk/~vgg/publications/2010/Everingham10/everingham10.pdf>.
- [6] Google, *Colab Frequently Asked Questions*, Resource Limits. Available at: <https://research.google.com/colaboratory/faq.html>.
- [7] CO3133, *Deep Learning and Its Applications*, Semester 261 course handbook, Sections 7.5, 18, 20, and 21.